Part (a): Initial EXPLAIN PLAN

Task: Alter the optimizer mode and generate the query plan for a large purchase search.

```
-- Set the optimizer settings as requested
ALTER SESSION SET OPTIMIZER_MODE = ALL_ROWS;
ALTER SESSION SET "_optimizer_cost_model"=CPU;

-- Clear any old plans from memory
DELETE FROM plan_table;

-- Generate the Execution Plan
EXPLAIN PLAN FOR
SELECT c.clno, c.name
FROM client c, purchase p
WHERE c.clno = p.clno AND p.qty > 1000;

-- Display the plan (Using standard Oracle DBMS display, alternative to utlxpls script)
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
SQL> -- Set the optimizer settings as requested
SQL> ALTER SESSION SET OPTIMIZER_MODE = ALL_ROWS;

Session altered.

SQL> ALTER SESSION SET "_optimizer_cost_model"=CPU;

Session altered.

SQL>
SQL> -- Clear any old plans from memory
SQL> DELETE FROM plan_table;

0 rows deleted.

SQL>
SQL> -- Generate the Execution Plan
SQL> EXPLAIN PLAN FOR
  2  SELECT c.clno, c.name
  3  FROM client c, purchase p
  4  WHERE c.clno = p.clno AND p.qty > 1000;

Explained.

SQL>
SQL> -- Display the plan (Using standard Oracle DBMS display, alternative to utlxpls script)
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 3129700840

-------------------------------------------------------------------------------
| Id  | Operation          | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |          |     6 |   138 |     4   (0)| 00:00:01 |
|*  1 |  HASH JOIN         |          |     6 |   138 |     4   (0)| 00:00:01 |
|   2 |   TABLE ACCESS FULL| CLIENT   |     2 |    30 |     2   (0)| 00:00:01 |
|*  3 |   TABLE ACCESS FULL| PURCHASE |     6 |    48 |     2   (0)| 00:00:01 |
-------------------------------------------------------------------------------


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("C"."CLNO"="P"."CLNO")
   3 - filter("P"."QTY">1000)

16 rows selected.
```

Part (b): Describe each step of the expected query plan

Explanation of the Un-indexed Plan: Before creating custom indexes, the Oracle Optimizer will typically choose a **TABLE ACCESS FULL** for the purchase table.

- Because there is no index on the qty column, the database engine is forced to scan every single row on the hard drive to check if qty > 1000.
- Once it filters those rows, it will perform a join (likely a **HASH JOIN** or **NESTED LOOPS**) with the client table to match the clno.

- Because reading full tables directly from disk is resource-intensive, the IO_COST (Input/Output Cost) and overall COST in this execution plan will be relatively high.

Part (c): Create Indexes

Task: Create unclustered B+ -Tree indexes on the purchase and client tables.

-- Create an index on qty and clno

CREATE INDEX idx_purch_qty_clno ON purchase(qty, clno);

-- Create an index on clno and name

CREATE INDEX idx_client_clno_name ON client(clno, name);

```
SQL>
SQL> -- Create an index on qty and clno
SQL> CREATE INDEX idx_purch_qty_clno ON purchase(qty, clno);

Index created.

SQL>
SQL> -- Create an index on clno and name
SQL> CREATE INDEX idx_client_clno_name ON client(clno, name);

Index created.
```

Part (d): Re-execute and Compare

Task: Run the EXPLAIN PLAN again and compare it to the original.

-- Clear old plan

DELETE FROM plan_table;

-- Generate the new Execution Plan

EXPLAIN PLAN FOR

SELECT c.clno, c.name

FROM client c, purchase p

WHERE c.clno = p.clno AND p.qty > 1000;

-- Display the updated plan

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

```
SQL> -- Clear old plan
SQL> DELETE FROM plan_table;

4 rows deleted.

SQL>
SQL> -- Generate the new Execution Plan
SQL> EXPLAIN PLAN FOR
  2  SELECT c.clno, c.name
  3  FROM client c, purchase p
  4  WHERE c.clno = p.clno AND p.qty > 1000;

Explained.
```

```
SQL>
SQL> -- Display the updated plan
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 2620475788

--------------------------------------------------------------------------------
----------

| Id  | Operation         | Name                | Rows  | Bytes | Cost (%CPU)|
Time     |

--------------------------------------------------------------------------------
----------


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |                     |     6 |   138 |     2   (0)|
00:00:01 |

|*  1 |  HASH JOIN        |                     |     6 |   138 |     2   (0)|
00:00:01 |

|   2 |   INDEX FULL SCAN | IDX_CLIENT_CLNO_NAME |     2 |    30 |     1   (0)|
00:00:01 |

|*  3 |   INDEX RANGE SCAN| IDX_PURCH_QTY_CLNO   |     6 |    48 |     1   (0)|
00:00:01 |

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
----------


Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("C"."CLNO"="P"."CLNO")
   3 - access("P"."QTY">1000 AND "P"."QTY" IS NOT NULL)


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Note
-----
   - this is an adaptive plan

20 rows selected.

SQL>
```

Comparison & Explanation of the New Query Plan: The new query plan is significantly more efficient than the plan in part (b) due to the creation of **Covering Indexes**.

- Every column requested by our query (c.clno, c.name, p.qty) is now stored directly inside the two new B+ Tree index structures.
- As a result, the Oracle Optimizer completely bypasses the physical tables. You will notice that **TABLE ACCESS FULL** has disappeared from the execution plan.
- Instead, Oracle performs an **INDEX RANGE SCAN** on idx_purch_qty_clno to instantly find quantities over 1000, and an **INDEX FAST FULL SCAN** or **INDEX UNIQUE SCAN** on idx_client_clno_name.
- Because searching an organized index in memory is much faster than scanning raw table data on a disk, the **CPU_COST**, **IO_COST**, and overall **COST** have dropped dramatically.